



Université Joseph Ki Zerbo
Système d'Information et Réseau - Semestre 1
Programmation Système

Enseignant : Dr. Tegawendé F. Bissyandé

25-27 Février 2023 / 3 Heures.

Veuillez indiquer votre nom au bas de chaque page.

Nom et Prénom :

Matricule :

Instructions : Répondre directement sur cette feuille dans les espaces prévus. Pour les QCM, veuillez noircir le(s) cercle(s) décrivant la ou les bonne(s) réponse(s). Chaque réponse incorrecte aux QCM entraînera une pénalité.

Part 1. QCM - Questions d'échauffements.

1. (2 points) Parmi ses rôles principaux, le système d'exploitation doit assurer :
 - (A) l'allocation du temps processeur aux processus en cours d'exécution ;
 - (B) la gestion de la concurrence, i.e. des accès cohérents aux données partagées par plusieurs processus ;
 - (C) l'édition des programmes et leur compilation.
2. (2 points) Un processus :
 - (A) est statique et le programme associé représente sa dynamique d'exécution ;
 - (B) représente l'image de l'état du processeur et de la mémoire au cours de l'exécution d'un programme ;
 - (C) la traduction d'un langage C en assembleur.
3. (2 points) La politique d'ordonnancement d'un système d'exploitation
 - (A) est la manière avec laquelle les fichiers physiques sont rangés sur le disque dur ;
 - (B) est la manière avec laquelle les droits des utilisateurs sont gérés les uns par rapport aux autres ;
 - (C) est la manière avec laquelle le temps processeur est attribué aux différents processus.
 - (D) est la manière avec laquelle les périphériques sont liés aux fichiers spéciaux.
4. (2 points) L'instruction C : `foo = getpid ()` ;
 - (A) place dans la variable `foo` l'identifiant du processus père du processus appelant ;
 - (B) place dans la variable `foo` le résultat de la création d'un processus ;
 - (C) place dans la variable `foo` l'adresse de la zone mémoire associé au processus appelant ;
 - (D) place dans la variable `foo` l'identifiant du processus appelant ;
 - (E) n'a pas d'impact sur le temps d'exécution du programme.

Nom et Prénom:



5. (2 points) Lors de la création d'un processus par un appel à `fork()` :
- (A) la terminaison du processus père n'entraîne pas la terminaison du processus fils ;
 - (B) les processus père et fils ne peuvent pas être synchronisés ;
 - (C) les processus père et fils peuvent être synchronisés en utilisant la fonction `wait()` qui prend un seul argument ;
 - (D) les processus père et fils peuvent être synchronisés en utilisant la fonction `waitpid()` qui ne prend pas d'argument.
6. (2 points) Dans le cas d'un processus léger (thread) :
- (A) l'accès à l'ensemble des variables du processus père est possible ;
 - (B) l'accès à l'ensemble des variables du processus père n'est pas possible.
7. (2 points) Un fichier spécial :
- (A) est en général lié à un périphérique ;
 - (B) peut être édité sans risque par un utilisateur ;
 - (C) ne permet jamais un accès direct ;
 - (D) permet, en fonction de sa nature, un accès direct.
8. (2 points) `perror()` :
- (A) ne sert à rien ;
 - (B) permet de traduire une erreur répertoriée en un message compréhensible par un humain ;
 - (C) est un outil de débogage au même titre que `valgrind` ;
 - (D) ne correspond à aucune des trois affirmations précédentes.
9. (2 points) La lecture dans un tube anonyme :
- (A) se fait de manière directe avec `read()` ;
 - (B) se fait de manière directe avec `fopen()` ;
 - (C) se fait de manière séquentielle avec `open()` .
10. (2 points) La redirection de sortie :
- (A) consister à rediriger la sortie standard vers la sortie d'un tube ;
 - (B) consister à rediriger l'entrée standard vers la sortie d'un tube ;
 - (C) consister à rediriger la sortie standard vers l'entrée d'un tube ;
 - (D) consister à rediriger l'entrée standard vers l'entrée d'un tube ;
 - (E) être réalisée dans le shell grâce à l'opérateur `&`.
11. (2 points) L'accès aux attributs associés à un fichier :
- (A) peut se faire avec la fonction `stat()` qui retourne un entier correspondant à la taille du fichier
 - (B) peut se faire en utilisant la fonction `stat()` qui modifie une structure ;
 - (C) est a priori utile si on souhaite recoder la fonction `ls`

Nom et Prénom:



12. (3 points) Qu'est-ce qu'un processus zombie ? Pourquoi a-t-on eu besoin d'introduire un tel "état" ?

13. (2 points) Quel risque-t-on lorsque le nombre de zombies devient grand ?

14. (4 points) Donnez un petit exemple de programme C provoquant cette situation à coup sûr.

Nom et Prénom:



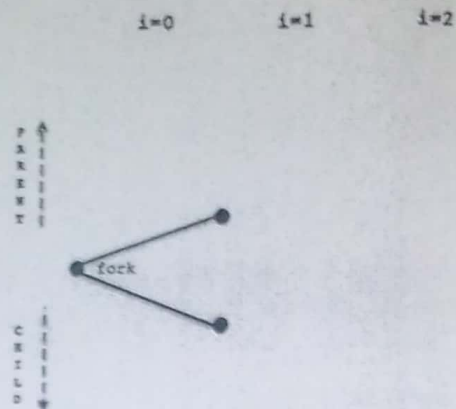
Part 2. Processus.

15. Considérez le programme suivant :

```
int main(){
    int i, status;

    for(i=0; i<3; i++){
        if(fork() == 0){
            /*child*/
            printf("Child: %d: Hello World!", i);
            _Exit(0);
        } else {
            /*parent*/
            printf("Parent: %d: Hello World!\n", i);
        }
    }

    wait(&status);
    if(WIFSIGNALED(status)){
        printf("Process signaled\n");
    }
    return 0;
}
```



- (a) (4 points) À droite du programme, compléter le dessin de l'arbre des processus ci-dessus, où chaque point représente un processus et un fork conduit à une séparation, vers le haut pour le père et vers le bas pour fils.
- (b) (6 points) Combien de processus au total sont créés, y compris le processus père initial ?

- (c) (5 points) Combien de processus tournent après la boucle for? Expliquer.

Nom et Prénoms:

**Part 3. Signaux.**

16. Considérez le programme suivant :

```
int ticks=0;
void handler(int signum){
    ticks++;
    printf("tick tock: %d\n", ticks);

    if(ticks < 10){
        alarm(1);
    }else{
        raise(9); //<-- MARK 1
    }
}

int main(){
    signal(SIGALRM, handler); //<-- MARK 2
    alarm(1);

    //MARK 3
    while(1){
        pause(); //<-- MARK 3
    }
}
```

- (a) (4 points) À MARK 2, quel est le but visé par l'appel à `signal()` concernant les prochaines livraisons de `SIGALRM` par le système d'exploitation ?

- (b) (3 points) Quel est le résultat de l'appel système à MARK 1 ? Quel est le nom du signal levé ?

Nom et Prénom:



(c) (4 points) Combien de signaux SIGALRM sont livrés par le programme ? Expliquez.

(d) (5 points) Quel est le rôle de `pause()` à MARK 3 ? Est-ce alors mieux ou pire de le remplacer avec le code suivant sans `pause()` ? `//MARK 3 while(1);`

Nom et Prénom:



Part 4. Communications par tubes.

17. On dispose d'un programme trier capable de trier un fichier d'entiers et d'envoyer le résultat (dans le même format) sur sa sortie standard. Les entiers sont stockés dans le même format que le format binaire natif du processeur. Pour afficher le résultat sous forme lisible, on utilise donc un programme afficher qui imprime sous forme décimale les entiers lus sur l'entrée standard.

```
> cat fichier | afficher
7 2 6 8 1
> trier fichier | afficher
1 2 6 7 8
```

trier.c

```
1 // La fonction suivante permet de trier un tableau d'entiers.
2 // On ne demande pas d'écrire son code.
3 extern void sort(int *array, unsigned nb_integers);
4
5 int main(int argc, char *argv[])
6 {
7     int fd;
8     unsigned len;
9     int *buf;
10    struct stat file_attributes;
11
12    fd = open(argv[1], O_RDONLY);
13
14    fstat (fd, &file_attributes);
15    len = file_attributes.st_size;
16
17    buf = malloc(len);
18    read(fd, buf, len);
19    sort(buf, len / sizeof(int));
20    write(STDOUT_FILENO, buf, len);
21
22    return 0;
23 }
```

- (a) (5 points) On souhaite modifier *trier* pour ne prendre en compte qu'une sous-partie du fichier. Pour cela, on ajoute deux arguments qui représentent respectivement la position du premier entier du segment à considérer et le nombre d'entiers à traiter. Voici deux exemples illustrant le nouveau fonctionnement de *trier* : Donnez le code remplaçant les lignes 14 et 15 afin de prendre en compte les deux nouveaux paramètres. On pourra utiliser les fonctions bien connues `off_t lseek(int fd, off_t offset, int whence)` (où `whence` prendra une valeur dans `SEEK SET`, `SEEK CUR`, `SEEK END`) et `int atoi(char *s)` pour transformer une chaîne de caractères en entier.

```
> trier fichier 0 3 | afficher
2 6 7
> trier fichier 3 2 | afficher
1 8
```

Nom et Prénom:



- (b) (5 points) Ecrivez un squelette de programme fusion créant deux processus fils, chacun connectés par un tube distinct au processus père. Les fils doivent uniquement pouvoir écrire dans le tube, et le père ne doit pouvoir que lire. Toutes les extrémités de tubes non utiles doivent être fermées. Pour cette question, les processus ne font aucun traitement utile : ils appellent exit sitôt la mise en place des tubes terminée.

Nom et Prénom: